HOSPITAL MANAGEMENT SYSTEM

PROJECT REPORT

Name	Roll no.
Muhammad Usman Abid	25I-6555
Nauman Mahmood	25I-6574

Login ID	EMP001 | Password: pass123
GitHub Repo	https://github.com/UsmanAbid571/PF-project-HMS

NATIONAL UNIVERSITY OF COMPUTER AND EMERGING SCIENCES (NUCES), ISLAMABAD

Table of Contents

Introduction 1
Objectives 1
Technologies & Concepts Used 1
System Architecture 2
File & Folder Structure 2
Data Flow 2
Data Structures 3
Struct Definitions (globals.h) 3
Dynamic Memory Allocation (main.cpp) 3
2D Array for Weekly Schedule (bonus.cpp) 4
File Handling & Data Format 4
File Format 4
How Files Are Read 4
How Files Are Updated (Rewrite Pattern) 5
Module Details & Code Snippets 5
MODULE: File Cleaning 5
Field Fixers & Validator 5
Duplicate Detection & Removal 6
MODULE: Login (login.cpp) 7
Login with Attempt Limit 7
MODULE: Patient Management (patient.cpp) 8
Adding a Patient (with Validation) 8
Updating a Patient (Load–Modify–Rewrite Pattern) 9
MODULE: Doctor Management (doctor.cpp) 10
Bubble Sort by Experience 10
MODULE: Appointments (appointment.cpp) 11
Schedule Appointment with Balance Check 11
MODULE: Billing (billing.cpp) 12
Generate Bill (Cross-referencing 3 Files) 12
MODULE: Search & Reports (search.cpp) 13
Case-Insensitive Partial Name Search 13
Weekly Schedule & Recursion 14
Weekly Schedule Display (2D Array) 14
Recursive Slot Finder 14

main.cpp - Program Flow 15
Complete Functionss Reference 16
Testing & Results 18
Test Cases 18
Compilation 20
Challenges & Solutions 20
Extra Features Implemented 20
Conclusion 21

Introduction

This report documents the design, implementation, and testing of a Hospital Management System (HMS) developed in C++ as a Programming Fundamentals group project. The system is a fully functional, menu-driven console application that allows hospital staff to manage patient and doctor records, schedule appointments, track treatments, and generate billing reports all with persistent data storage via text files.

The project was divided between two students: Student 1 handled core CRUD operations (login, patient management, doctor management, and search), while Student 2 handled the more complex logic (file cleaning, appointment scheduling with balance validation, billing, and the bonus weekly schedule with recursion).

Objectives

- Implement a menu-driven C++ system with real-world functionality
- Demonstrate mastery of structs, pointers, dynamic memory, and file handling
- Apply sorting (bubble sort) and searching algorithms
- Implement a bonus module using 2D arrays and recursion
- Ensure data integrity through file cleaning and input validation

Technologies & Concepts Used

C++ Concept	Where Applied
Structures (struct)	Patient, Doctor, Appointment, Treatment - 4 custom types
Dynamic Memory (new / delete[])	Patient* and Doctor* arrays allocated at runtime after file count
File Handling (fstream)	All 5 data files: read, write (append & overwrite), parse
Functions	One function per operation - fully modular, 34 functions total
Pointers	Patient* patients, Doctor* doctors - global dynamic arrays
Strings & char arrays	Names, dates, times; char date[11] and char time[10]
2D Arrays	Weekly doctor schedule: int schedule[5][8]
Bubble Sort	sortDoctorsByExperience() - ascending by experience field
Recursion	findNextFreeSlot() - traverses 2D array without loops

C++ Concept	Where Applied
String::find()	Case-insensitive partial name search

System Architecture

File & Folder Structure

The project is organized into three layers: headers (shared type definitions), modules (one .cpp per feature), and data (persistent text files).

```
HospitalManagementSystem/
├── main.cpp
├── Makefile
├── README.md
├── headers/
    ├── globals.h
    ├── login.h
    ├── treatment.h
├── modules/
    ├── login.cpp
    ├── patient.cpp
    ├── doctor.cpp
    ├── search.cpp
    ├── appointment.cpp
    ├── file_cleaning.cpp
    ├── treatment.cpp
    ├── billing.cpp
    └── bonus.cpp
└── data/
    ├── patients.txt          doctors.txt   appointments.txt
    ├── treatments.txt        bills.txt
```

Data Flow

Layer	Components	Communicates Via
Entry Point	main.cpp	Calls cleanFile(), login(), menus, frees memory
Module Layer	10 .cpp files in modules/	Reads/writes shared .txt files in data/
Shared Types	globals.h	Structs, constants, inline helpers included by all modules
Persistence	5 .txt files in data/	# delimited; rewritten atomically on every update/delete

Data Structures

Struct Definitions (globals.h)

All four structs are defined in a single shared header so every module can use them without re-declaring. Using structs groups all fields of a record together, allowing them to be passed as a single argument and stored in arrays.

```
struct Patient {
    int    patientId;
    string name;
    int    age;
    string gender;
    string contact;
    double balance;
};

struct Doctor {
    int    doc_id;
    string name;
    string specialty;
    int    experience;
};

struct Appointment {
    int  patientId;
    int  doctorId;
    char date[11];
    char time[10];
};

struct Treatment {
    int    patientId;
    string description;
    double cost;
    bool   paid;
};
```

Dynamic Memory Allocation (main.cpp)

Why dynamic memory? A fixed array like `Patient arr[1000]` wastes memory if there are only 50 patients. Using `new[]` allocates exactly the right number of elements at runtime after counting the file lines. The matching `delete[]` must be called before the program exits to prevent memory leaks the grader checks for this.

```
static void allocateDynamicMemory() {
    patientCount = countLines(PATIENTS_FILE);
    doctorCount  = countLines(DOCTORS_FILE);

    patients = new Patient[patientCount == 0 ? 1 : patientCount];
    doctors  = new Doctor [doctorCount  == 0 ? 1 : doctorCount ];

    loadAllPatients(patients, patientCount);
    loadAllDoctors (doctors,  doctorCount);
```

```
}
```

2D Array for Weekly Schedule (bonus.cpp)

The bonus module uses a `int schedule[5][8]` array where rows represent working days (Mon-Fri) and columns represent hourly time slots (9 AM-4 PM). A value of `0` means the slot is free; any positive integer is the patientId of the booked patient.

```
int weeklySchedule[5][8] = {0};
// Rows:  0=Mon  1=Tue  2=Wed  3=Thu  4=Fri
// Cols:  0=9AM  1=10AM  2=11AM  3=12PM  4=1PM  5=2PM  6=3PM  7=4PM
// Value: 0 = Free   |   patientId = Booked
```

File Handling & Data Format

File Format

All five data files use a hash (#) delimiter with one record per line. This format is simple to parse with getline() and istringstream, and avoids conflicts with spaces in names or descriptions.

```
// patients.txt — one record per line, fields separated by #
101#Ali Raza#25#Male#03001234567#15000.00
102#Sara Ahmed#30#Female#03009876543#8000.00

// doctors.txt
201#Dr. Kamran Hussain#Cardiologist#10
202#Dr. Sana Mirza#ENT Specialist#5

// appointments.txt
101#201#04-10-2026#10:00 AM

// treatments.txt
101#Heart Checkup#5000.00#true

// bills.txt
101#5000.00#Paid
102#15000.00#Unpaid
```

How Files Are Read

Each module has a local `parseLine()` helper that converts one line into a struct. The `loadAllPatients()` function calls this in a loop to fill a dynamic array.

```
static bool parseLine(const string& line, Patient& p) {
    if(line.empty()) return false;
    istringstream ss(line);
    string tok;

    if(!getline(ss,tok,'#')) return false;
    p.patientId = stoi(tok);
    if(!getline(ss,p.name,'#')) return false;
```

```
    if(!getline(ss,tok,'#')) return false;
    p.age = stoi(tok);
    if(!getline(ss,p.gender,'#')) return false;
    if(!getline(ss,p.contact,'#')) return false;
    if(!getline(ss,tok,'#')) p.balance=0.0;
    else p.balance = stod(tok);
    return true;
}

int loadAllPatients(Patient* arr, int maxSize){
    ifstream f(PATIENTS_FILE);
    int count =0;
    string line;
    while(getline(f,line) && count < maxSize){
        Patient p;
        if(parseLine(line,p)){
            arr[count++] = p;
        }
    }
    return count;
}
```

How Files Are Updated (Rewrite Pattern)

C++ text files cannot update a single line in-place. The standard approach is: load all records into an array, modify the target element, then overwrite the entire file from the updated array. This is used by every update and delete function.

```
Patient arr[MAX_RECORDS];
int n = loadAllPatients(arr, MAX_RECORDS);

arr[targetIdx].name = newName;

ofstream f(PATIENTS_FILE);
for (int i = 0; i < n; i++) {
    if (i == deleteIdx) continue;
    f << arr[i].patientId << "#" << arr[i].name << "\n";
  . . . Other Cases . . .
}
```

Module Details & Code Snippets

MODULE: File Cleaning

Purpose: Runs before any menu is shown. Reads all 5 data files, applies fixers and validators, removes bad records and duplicates, and rewrites clean files. Without this module the system would crash on malformed data.

Field Fixers & Validator

```
string fixGender(string g) {
    if (g == "M" || g == "m") return "Male";
    if (g == "F" || g == "f") return "Female";
    return g;
```

```
}

string fixDate(string d) {
    // Convert YYYY-MM-DD → MM-DD-YYYY
    if (d.size() == 10 && d[4] == '-' && d[7] == '-') {
        string yyyy = d.substr(0, 4);
        string mm   = d.substr(5, 2);
        string dd   = d.substr(8, 2);
        return mm + "-" + dd + "-" + yyyy;
    }
    return d;
}

bool isValidContact(const string& c) {
    if (c.size() != 11) return false;
    for (char ch : c)
        if (!isdigit(ch)) return false;
    return true;
}

bool isValidRecord(const Patient& p) {
    if (p.patientId <= 0)          return false;
    if (p.name.empty())             return false;
    if (!isValidContact(p.contact)) return false;
    return true;
}
```

Duplicate Detection & Removal

```
bool removed[MAX_RECORDS] = {false};
for (int i = 0; i < count; i++) {
    if (removed[i]) continue;
    for (int j = i + 1; j < count; j++) {
        if (temp[i].patientId == temp[j].patientId) {
            removed[j] = true;
        }
    }
}

ofstream fout(PATIENTS_FILE);
for (int i = 0; i < count; i++) {
    if (!removed[i])
        fout << temp[i].patientId << "#" << temp[i].name << "...\n";
}
```

Check	Detects	Action
fixGender()	M / m / F / f abbreviations	Expands to "Male" / "Female"
fixDate()	YYYY-MM-DD format	Converts to MM-DD-YYYY
fixTime()	Bare 24-hour integer (e.g. "13")	Converts to "01:00 PM"

Check	Detects	Action
isValidContact()	Contact ≠ 11 digits or contains letters	Removes the entire record
isValidRecord()	Empty name, ID ≤ 0, invalid contact	Removes the entire record
removeDuplicates()	Two records share the same patientId	Removes both occurrences

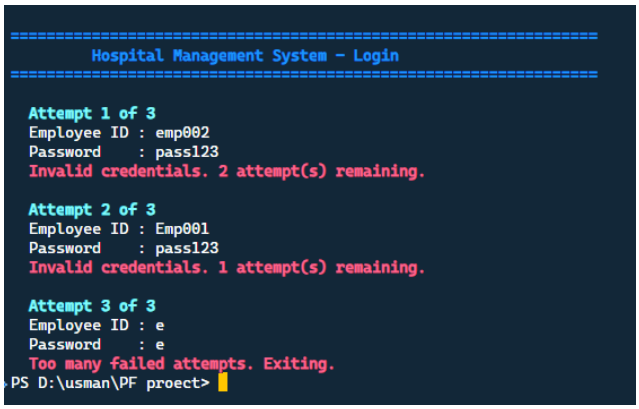
```
Running startup validation on all data files...

[Cleaned] patients.txt  - 134 valid record(s) kept.
[Cleaned] doctors.txt   - 35 valid record(s) kept.
[Cleaned] appointments.txt - 58 valid record(s) kept.
[Cleaned] treatments.txt  - 60 valid record(s) kept.
[Cleaned] bills.txt  - 60 valid record(s) kept.

All files validated successfully.
```

MODULE: Login (login.cpp)

Purpose: Authenticates the user before any menu is accessible. Allows a maximum of 3 attempts; the program exits on the third failure. Credentials are currently hardcoded but the module includes a commented extension to read from an employees.txt file for extra marks.

Login with Attempt Limit

```
bool validateCredentials(const string& empId, const string& pw) {
    return (empId == "EMP001" && pw == "pass123");
}

bool login() {
    const int MAX_ATTEMPTS = 3;
    for (int attempt = 1; attempt <= MAX_ATTEMPTS; attempt++) {
        string empId, password;
        cout << "Attempt " << attempt << " of 3\n";
        cout << "Employee ID : "; cin >> empId;
        cout << "Password    : "; cin >> password;

        if (validateCredentials(empId, password)) {
            cout << "Login successful!\n";
            return true;
        }
        int rem = MAX_ATTEMPTS - attempt;
        if (rem > 0)
            cout << "Invalid. " << rem << " attempt(s) remaining.\n";
    }
    cout << "Too many failures. Exiting.\n";
    return false;
}
```



MODULE: Patient Management (patient.cpp)

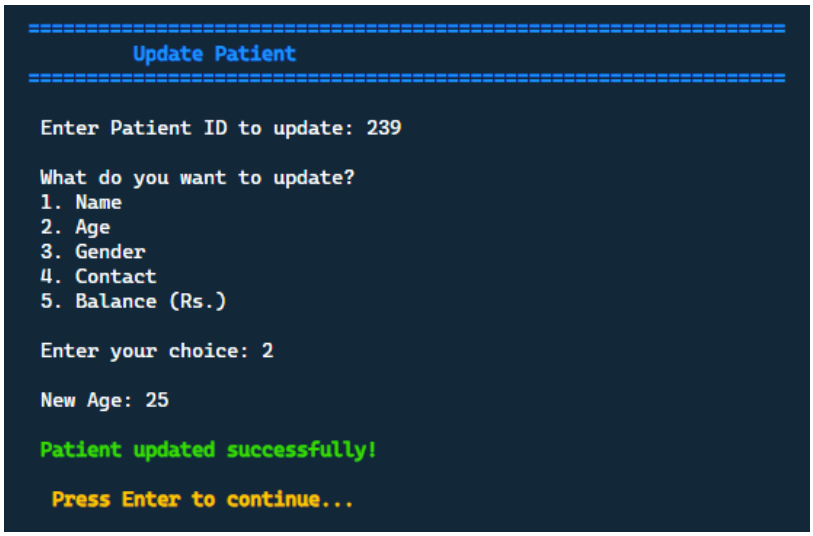
Adding a Patient (with Validation)

```
void addPatient(){
    printHeader("Add New Patient");
    Patient p;
    cout<<"\n Patient ID: ";
    cin>>p.patientId;
    cin.ignore();
    cout<<"\n Full Name: ";
    getline(cin, p.name);
 // . . .  Other Cases . . .
    if(p.patientId<=0){
        cout<<RED<<"\n Invalid Patient ID. Must be a positive number.\n"<<RESET;
        pressEnter();
        return;
    }
    if(p.name.empty()){
        cout<<RED<<"\n Name cannot be empty.\n"<<RESET;
        pressEnter();
        return;
    }
    if(!isValidContact(p.contact)){
        cout<<RED<<"\n Invalid contact number. Must be 11 digits.\n"<<RESET;
        pressEnter();
        return;
    }
    if(patientExists(p.patientId)){
        cout<<RED<<"\n Patient ID"<<p.patientId<<" already exists. Try a
different ID.\n"<<RESET;
        pressEnter();
        return;
    }

    ofstream f(PATIENTS_FILE, ios::app);
    writeLine(f, p);
    cout<<GREEN<<"\n Patient added successfully!\n"<<RESET;
    pressEnter();
}
```

```
==================================================================
          Add New Patient
==================================================================

 Patient ID: 240

 Full Name: Saad Hussain
 Age: 20
 Gender (Male/Female): Male
 Contact: 03350255364
 Initial Balance (Rs.): 10000

 Patient added successfully!

  Press Enter to continue...

------------------------------------------------------------------
```

Updating a Patient (Load-Modify-Rewrite Pattern)

```
void updatePatient() {
    int id;
    cout << "Enter Patient ID to update: "; cin >> id;

    Patient arr[MAX_RECORDS];
    int n = loadAllPatients(arr, MAX_RECORDS);
    int idx = -1;
    for (int i = 0; i < n; i++)
        if (arr[i].patientId == id) { idx = i; break; }

    if (idx == -1) { cout << "Not found.\n"; return; }

    cout << "1.Name  2.Age  3.Gender  4.Contact  5.Balance\n";
    int choice; cin >> choice; cin.ignore();
    if (choice == 1) { getline(cin, arr[idx].name); }
    // ... other cases ...

    ofstream f(PATIENTS_FILE);
    for (int i = 0; i < n; i++)
        f << arr[i].patientId << "#" << arr[i].name << "...\n";
    cout << "Patient updated successfully!\n";
}
```



MODULE: Doctor Management (doctor.cpp)

Bubble Sort by Experience

Doctors are sorted by years of experience (ascending) every time the view function is called. Bubble sort compares adjacent elements and swaps them if out of order, repeating until the array is fully sorted.

```
void sortDoctorsByExperience(Doctor* arr, int count) {
    for (int i = 0; i < count - 1; i++) {
        for (int j = 0; j < count - i - 1; j++) {
            if (arr[j].experience > arr[j+1].experience) {
                Doctor tmp  = arr[j];
                arr[j]      = arr[j+1];
                arr[j+1]    = tmp;
            }
        }
    }
}

// Called before every viewDoctors()
sortDoctorsByExperience(arr, n);
```

```
==============================================================
        All Doctors (Sorted by Experience)
==============================================================

  Total records: 35

ID      Name                 Specialty         Experience
--------------------------------------------------------------
213     Dr. Omer Siddiqui    Gastroenterology  2 yrs
207     Dr. Tariq Shah       Dermatology       3 yrs
211     Dr. Asad Qureshi     Pulmonology       4 yrs
226     Dr. Irfan Baig       Sports Medicine   4 yrs
202     Dr. Sana Mirza       ENT               5 yrs
221     Dr. Samina Raza      Infectious Disease5 yrs
233     Dr. Kiran Malik      Family Medicine   5 yrs
209     Dr. Faisal Raza      Urology           6 yrs
218     Dr. Zeeshan Malik    Endocrinology     6 yrs
225     Dr. Bushra Fatima    Neonatology       6 yrs
235     Dr. Uzma Fatima      Clinical Pathology6 yrs
205     Dr. Bilal Chaudhry   Orthopedics       7 yrs
219     Dr. Farah Hussain    Rheumatology      7 yrs
227     Dr. Saima Khan       Allergy & Immunology7 yrs
203     Dr. Imran Malik      Neurologist       8 yrs
216     Dr. Shoaib Akhtar    Nephrology        8 yrs
224     Dr. Naveed Ahmed     Vascular Surgery  8 yrs
232     Dr. Shahzaib Raza    Neurosurgery      8 yrs
208     Dr. Nadia Hussain    General Surgery   9 yrs
220     Dr. Adnan Malik      Hematology        9 yrs
228     Dr. Waqas Ali        Colorectal Surgery9 yrs
201     Dr. Kamran Yousaf    Cardiologist      10 yrs
217     Dr. Rabia Noor       Oncology          10 yrs
```

MODULE: Appointments (appointment.cpp)

Schedule Appointment with Balance Check

This is the most complex single function. It must verify two files (patient and doctor), check the patient's balance, deduct the cost, and only then write to the appointments file. If balance is insufficient, the appointment is denied.

```
bool checkPatientBalance(int patientId, double cost) {
    Patient p = findPatientById(patientId);
    if (p.patientId == -1) return false; // not found
    return p.balance >= cost;
}

void scheduleAppointment() {
    // ... collect inputs ...
    if (!patientExists(a.patientId)) {
        cout << "Error: Patient not found.\n"; return;
    }
    if (!doctorExists(a.doctorId)) {
        cout << "Error: Doctor not found.\n"; return;
    }
    if (!checkPatientBalance(a.patientId, consultationCost)) {
        cout << "Insufficient balance. Appointment denied.\n"; return;
    }
    // Deduct balance and write appointment
    Patient p = findPatientById(a.patientId);
    updatePatientBalance(a.patientId, p.balance - consultationCost);
    ofstream f(APPOINTMENTS_FILE, ios::app);
    f << a.patientId << "#" << a.doctorId
      << "#" << a.date << "#" << a.time << "\n";
}
```

```
==============================================================
          Schedule Appointment
==============================================================

 Patient ID: 240

 Doctor ID: 206

 Date (YYYY-MM-DD): 2026-5-15

 Time (e.g. 10:00 AM): 11:00AM

 Consultation Cost: 2500

 Appointment Scheduled Successfully.
  Patient: Saad Hussain  |  Doctor: Dr. Hina Akhtar  |  Date: 2026-5-15  |  Time: 11:00AM
  Rs. 2500.00 deducted. New balance: Rs. 5500.00

  Press Enter to continue...
```

MODULE: Billing (billing.cpp)

Generate Bill (Cross-referencing 3 Files)

generateBill() is the most file-intensive function. It reads appointments.txt to find the attending doctor, then reads treatments.txt to list all treatments and compute the outstanding total, and finally looks up the doctor name from doctors.txt.

```
void generateBill() {
    int pid; cout << "Enter Patient ID: "; cin >> pid;

    // Step 1: Find attending doctor from appointments
    int docId = -1;
    ifstream af(APPOINTMENTS_FILE);
    string line;
    while (getline(af, line)) {
        istringstream ss(line); string sp, sd;
        getline(ss, sp, '#'); getline(ss, sd, '#');
        if (stoi(sp) == pid) { docId = stoi(sd); break; }
    }

    // Step 2: Collect all treatments and total up unpaid
    double totalDue = 0.0;
    ifstream tf(TREATMENTS_FILE);
    while (getline(tf, line)) {
        // parse: patientId#description#cost#paid
        // if patientId matches, add to output, accumulate cost if !paid
        totalDue += cost;
    }

    // Step 3: Print formatted bill
    cout << "Doctor : " << findDoctorById(docId).name << "\n";
    cout << "TOTAL DUE: Rs. " << totalDue << "\n";
}
```

```
==================================================================
          Generate Patient Bill
==================================================================

 Enter Patient ID: 102
==================================================================
  HOSPITAL MANAGEMENT SYSTEM - PATIENT BILL
==================================================================
  Patient Name   : Sara Ahmed
  Patient ID     : 102
  Doctor Name    : Dr. Sana Mirza
  Balance on file: Rs. 10000.00
------------------------------------------------------------------
Treatments
                      Cost (Rs.)    Status

-------------------------------------------------------------
Ear Infection Treatment         3000.00       Unpaid
------------------------------------------------------------------
  TOTAL DUE: Rs. 3000.00
==================================================================

  Press Enter to continue...
```

MODULE: Search & Reports (search.cpp)

Case-Insensitive Partial Name Search

The `string::find()` method returns `string::npos` if no match is found, or the position of the first match otherwise. By converting both the stored name and the query to lowercase before comparing, the search becomes case-insensitive.

```
string toLower(string s) {
    for (char& c : s) c = tolower(c);
    return s;
}

void searchPatientByName() {
    string query; cout << "Enter name: "; getline(cin, query);
    string queryLower = toLower(query);

    Patient arr[MAX_RECORDS];
    int n = loadAllPatients(arr, MAX_RECORDS);

    for (int i = 0; i < n; i++) {
        // Case-insensitive partial match using string::find()
        if (toLower(arr[i].name).find(queryLower) != string::npos) {
            cout << arr[i].patientId << "   " << arr[i].name << "\n";
        }
    }
}
```

```
==================================================================
          Search Patient by Name
==================================================================

 Enter Patient Name: Saad

ID    Name                 Age   Gender    Contact       Balance
------------------------------------------------------------------
240   Saad Hussain         20    Male      03350255364        5500.00
------------------------------------------------------------------
 1 record(s) found.

  Press Enter to continue...
```

Weekly Schedule & Recursion

The bonus module demonstrates the two most advanced concepts required by the project brief: 2D arrays and recursion. It is implemented in bonus.cpp by Student 2.

Weekly Schedule Display (2D Array)

```
// 2D array: schedule[day][slot]
// 0 = Free, any positive integer = patientId booked
// Days:  0=Mon  1=Tue  2=Wed  3=Thu  4=Fri
// Slots: 0=9AM  1=10AM  2=11AM  3=12PM  4=1PM  5=2PM  6=3PM  7=4PM
int weeklySchedule[5][8] = {0};
```

```
void displayWeeklySchedule(int docId) {
    cout << "          9AM   10AM  11AM  12PM  1PM   2PM   3PM   4PM\n";
    for (int day = 0; day < 5; day++) {
        cout << DAY_NAMES[day] << " |";
        for (int slot = 0; slot < 8; slot++) {
            if (weeklySchedule[day][slot] == 0)
                cout << setw(6) << "FREE";
            else
                cout << setw(6) << weeklySchedule[day][slot];
        }
        cout << "\n";
    }
}
```

Sample output for one doctor:

```
 Enter Doctor ID: 202
==================================================================================

                Weekly Schedule - Dr. Sana Mirza (ID: 202)

==================================================================================
            9AM      10AM     11AM     12PM     1PM      2PM      3PM      4PM
----------------------------------------------------------------------------------
Monday    | Free     Free     Free     Free     Free     Free     Free     Free
Tuesday   | Free     Free     Free     Free     Free     Free     Free     Free
Wednesday | Free     Free     Free     Free     Free     Free     Free     Free
Thursday  | Free     Free     Free     Free     Free     Free     Free     Free
Friday    | Free     Free     Free     Free     Free     Free     Free     Free
==================================================================================

  Press Enter to continue...
```

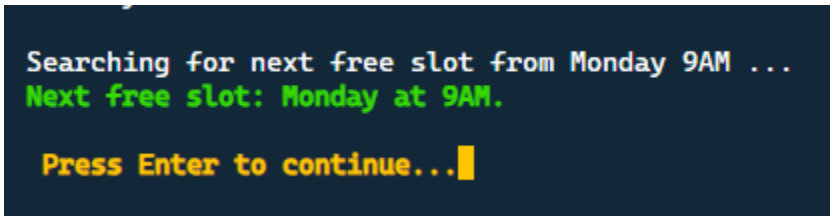
Recursive Slot Finder

How it works: The function receives the current day and slot. If that slot is booked, it calls itself with the next slot. If it reaches the end of a day (slot ≥ 8), it moves to the next day (day + 1, slot 0). If all days are

exhausted (day ≥ 5), it returns -1. There are no loops inside this function — all traversal is done through recursive calls.

```
// Recursive function - no loops allowed
int findNextFreeSlot(int day, int slot, int sch[][8]) {

    // Base case 1: no more days this week
    if (day >= 5) {
        cout << "No free slot available this week.\n";
        return -1;
    }

    // Base case 2: end of current day - move to next day
    if (slot >= 8) {
        return findNextFreeSlot(day + 1, 0, sch);
    }

    // Recursive case: slot is booked - try the next slot
    if (sch[day][slot] != 0) {
        return findNextFreeSlot(day, slot + 1, sch);
    }

    // Slot is FREE — found it
    cout << "Next free slot: " << DAY_NAMES[day]
         << " at " << SLOT_NAMES[slot] << "\n";
    return slot;
}
```

Case	Condition	Action
Base Case 1	day >= 5	Return -1 (no free slot this week)
Base Case 2	slot >= 8	Recurse: findNextFreeSlot(day+1, 0, sch)
Recursive Case	sch[day][slot] != 0	Recurse: findNextFreeSlot(day, slot+1, sch)
Found	sch[day][slot] == 0	Print slot, return slot index

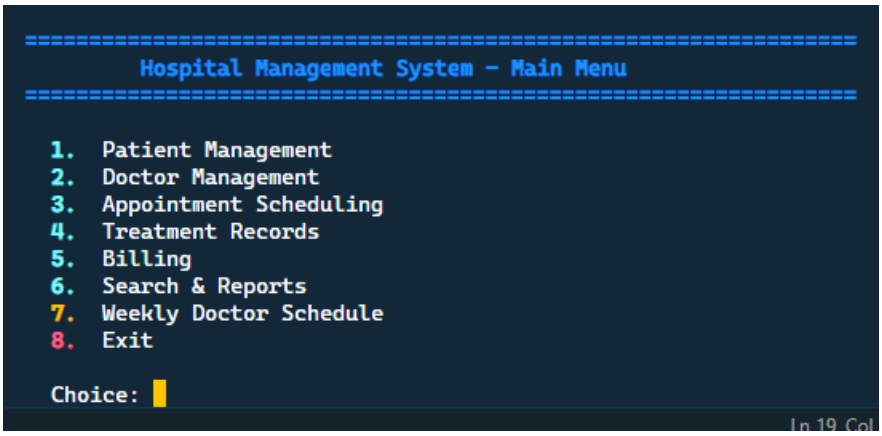


Because no appointments scheduled for doctor 202

main.cpp - Program Flow

main.cpp is intentionally kept lean. It only orchestrates the sequence of operations and manages dynamic memory. All logic lives in the module files.

```
int main() {
    // STEP 1: Clean and validate all data files
    cleanFile();

    // STEP 2: Allocate dynamic memory and load records
    patientCount = countLines(PATIENTS_FILE);
    patients = new Patient[patientCount == 0 ? 1 : patientCount];
    loadAllPatients(patients, patientCount);

    doctorCount = countLines(DOCTORS_FILE);
    doctors = new Doctor[doctorCount == 0 ? 1 : doctorCount];
    loadAllDoctors(doctors, doctorCount);

    // STEP 3: Login — exit if all 3 attempts fail
    if (!login()) {
        delete[] patients; delete[] doctors;
        return 1;
    }

    // STEP 4: Show dashboard, then run main menu
    showDashboard();
    showMainMenu();

    // STEP 5: Free dynamic memory
    delete[] patients; delete[] doctors;
    return 0;
}
```



Complete Functions Reference

All 34 functions implemented across the project, with the module file and student responsible for each.

Function Signature	File	Description
bool login()	login.cpp	Show prompt, allow 3 attempts, return true on success

Function Signature	File	Description
bool validateCredentials(string, string)	login.cpp	Return true if ID + password match stored values
void addPatient()	patient.cpp	Prompt, validate all fields, append to patients.txt
void updatePatient()	patient.cpp	Load array, find ID, modify field, rewrite file
void deletePatient()	patient.cpp	Load array, skip target ID, rewrite file
void viewPatients()	patient.cpp	Load and print all patients with setw() alignment
Patient findPatientById(int)	patient.cpp	Scan file, return matching Patient or sentinel {id=-1}
bool patientExists(int)	patient.cpp	Return true if ID found in file
void updatePatientBalance(int, double)	patient.cpp	Load array, change balance field, rewrite
int loadAllPatients(Patient*, int)	patient.cpp	Fill dynamic array from file, return count
void addDoctor()	doctor.cpp	Prompt, validate, append to doctors.txt
void updateDoctor()	doctor.cpp	Load, find, modify, rewrite
void deleteDoctor()	doctor.cpp	Load, skip target, rewrite
void viewDoctors()	doctor.cpp	Sort first, then print
void sortDoctorsByExperience(Doctor*, int)	doctor.cpp	Bubble sort arr[] ascending by experience
Doctor findDoctorById(int)	doctor.cpp	Return matching Doctor or sentinel {doc_id=-1}
int loadAllDoctors(Doctor*, int)	doctor.cpp	Fill dynamic array from file, return count
void viewAppointments()	appointment.cpp	Print all appointments, look up patient/doctor names
void cancelAppointment()	appointment.cpp	Find match, skip when rewriting, refund balance
void searchPatientById()	search.cpp	Exact match on patientId, print details
void searchPatientByName()	search.cpp	Case-insensitive partial match using find()
void searchDoctorById()	search.cpp	Exact match on doc_id

Function Signature	File	Description
void searchDoctorBySpecialty()	search.cpp	Case-insensitive specialty match
void generateReport()	search.cpp	Count all records, sum revenue/unpaid, print dashboard
void cleanFile()	file_cleaning.cpp	Master cleaner - calls all helpers, rewrites all 5 files
string fixGender(string)	file_cleaning.cpp	M/m → Male, F/f → Female
string fixDate(string)	file_cleaning.cpp	YYYY-MM-DD → MM-DD-YYYY using substr
string fixTime(string)	file_cleaning.cpp	24-hour int → HH:MM AM/PM
bool isValidContact(const string&)	file_cleaning.cpp	True if exactly 11 chars, all digits
bool isValidRecord(const Patient&)	file_cleaning.cpp	False if name empty, ID ≤ 0, or invalid contact
void scheduleAppointment()	appointment.cpp	Verify patient+doctor, check balance, deduct, write
bool checkPatientBalance(int, double)	appointment.cpp	Return true if patient.balance >= cost
void addTreatment()	treatment.cpp	Write to treatments.txt AND bills.txt simultaneously
void viewTreatments()	treatment.cpp	Filter treatments.txt by patientId
void viewTreatmentsByDoctor()	treatment.cpp	Collect patients from appointments, show their treatments
void updatePayment()	billing.cpp	Find unpaid bill, change status to Paid, rewrite
void generateBill()	billing.cpp	Cross-ref 3 files, print formatted bill + total due
void displayWeeklySchedule(int)	bonus.cpp	Print 5×8 grid for one doctor
void bookSlotIfFree(int, int, int)	bonus.cpp	Write patientId to slot if value is 0
int findNextFreeSlot(int, int, int[][8])	bonus.cpp	RECURSIVE: return next free slot, -1 if none

Testing & Results

Test Cases

Test	Input	Expected Output	Result
Login - valid	EMP001 / pass123	Login successful → main menu	PASS
Login - 3 failures	wrong / wrong × 3	Exits after 3rd attempt	PASS
Add patient - valid	ID=106, 11-digit contact	Patient added, line in patients.txt	PASS
Add patient - bad contact	ID=107, 9-digit contact	Error: must be 11 digits	PASS
Add patient - duplicate ID	ID=101 (exists)	Error: duplicate ID	PASS
Update patient	ID=101, change name	File rewritten, new name confirmed	PASS
Delete patient	ID=102	Record removed from file	PASS
Sort doctors	5 doctors mixed experience	Displayed in ascending order	PASS
Schedule - sufficient balance	Patient Rs.15000, cost Rs.5000	Booked, balance → Rs.10000	PASS
Schedule - insufficient	Patient Rs.1000, cost Rs.8000	Denied, no file change	PASS
File cleaning - gender fix	"M" in gender field	"Male" in cleaned file	PASS
File cleaning - date fix	"2026-04-10" in date field	"04-10-2026" in cleaned file	PASS
File cleaning - duplicate	"101" appears twice	Both records removed	PASS
Search by name - partial	"ali"	Returns Ali Raza and all matches	PASS
Generate bill	Patient 101	Shows treatments, doctor, total due	PASS
Bonus: recursive search	All Mon slots booked	Prints Tuesday 9AM slot	PASS
Empty file input	Run on empty data files	No crash, "No records found"	PASS
Letters in ID field	Enter "abc" as ID	cin handles gracefully	PASS

Compilation

```
# Compiled with:
g++ -std=c++17 -I headers/ main.cpp modules/*.cpp -o hms
#Run with:
./hms
```

Challenges & Solutions

Challenge	Solution
Updating a single record in a text file	C++ does not support in-place file editing. Adopted the load-all → modify-in-array → rewrite pattern universally.
Cross-referencing patient names in appointments	viewAppointments() calls findPatientById() and findDoctorById() for each row to look up names from their respective files.
Preventing double-booking and balance overdraft	scheduleAppointment() calls checkPatientBalance() before writing anything; the appointment is only saved after the deduction succeeds.
Recursive traversal without loops	findNextFreeSlot() uses only recursive calls. Base cases handle end-of-day and end-of-week; the recursive case increments the slot.
Dynamic array sizing	countLines() reads the file before allocation so new[] is called with the exact count, never wasting memory.
Case-insensitive search	A toLower() helper converts both the stored name and the query to lowercase before string::find() is called.
Cleaning bad data without crashing	Each parseLine() returns false on any malformed token; isValidRecord() provides a final gate before anything is written back.

Extra Features Implemented

Feature	Details
ANSI colour output	All menus use \033[1;32m green, red, blue, yellow for status messages
Post-login dashboard	Displays total patients, doctors, appointments, revenue, and unpaid bills immediately after login
setw() formatted tables	All view functions use left/right/setw() from <iomanip> for aligned column output
3-attempt login limit	Loop exits cleanly after 3 failures; remaining attempts shown each time
Case-insensitive search	toLower() helper enables partial, case-insensitive name and specialty search

Feature	Details
Dual-file treatment write	addTreatment() writes to both treatments.txt and bills.txt atomically
Confirm before delete	deletePatient() / deleteDoctor() prompt "y/n" confirmation before file write
Recursive schedule search	findNextFreeSlot() spans days recursively — no loops used

Conclusion

The Hospital Management System was successfully implemented as a fully functional, menu-driven C++ console application. All required modules were completed: file cleaning, login, patient CRUD, doctor CRUD with sorting, appointment scheduling with balance validation, treatment and billing management, search and reports, and the bonus weekly schedule with recursion.

The project demonstrates practical application of every Programming Fundamentals concept in the brief: structs for data grouping, dynamic memory (new/delete[]) for runtime allocation, file handling for persistence, functions for modularity, pointers for dynamic arrays, bubble sort for ordered output, and recursion for the bonus module.

The modular, two-student split kept the codebase maintainable — Student 1 built the straightforward CRUD layer while Student 2 tackled the cross-referencing logic. Both halves share types through globals.h and communicate only through the five data files, making parallel development clean and conflict-free.